

A] Bug 1: SQL AND/OR precedence bug

1) Where $\rightarrow$ File: TaskRepository.java
layer: ~~Java~~ SQL

2) How did I found it $\rightarrow$ I read the query and saw AND and OR mixed with no brackets.

Search q=api it gives 10 results with archived tasks also.

3) Root cause $\rightarrow$ SQL query uses both AND and OR without grouping the title and description conditions.

e.g. "I want a cold AND (tea OR coffee)"
without brackets = cold AND tea OR coffee
it read as (cold AND tea) OR coffee
so I ~~always~~ get hot coffee.

It always does AND before OR.

4) How I fixed it $\rightarrow$ I added parentheses around the title and description search conditions:

- archived = FALSE AND (title matches OR description matches) AND status matches

This makes sure that archived tasks are always excluded and status filter apply correctly.
After fixing it gives 8 results becoz 2 are archived ones.

5) Before and After adding brackets $\rightarrow$

Before: archived = FALSE AND title match OR description match AND status

After: archived = FALSE AND (title match OR description match) AND status

B) Bug 2: Artificial delay (Thread.sleep)

1) where → File: TaskController.java

layer: Backend

2) How did I found it → I read TaskController.java and saw Thread.sleep(). where its comment that saying "query complexity for logging". In testing I actually saw the slowness.

3) Root cause → The delay cause: formula = $(10 - \text{length of search}) * 100 \text{ ms}$.

So the empty search waited 1000 ms which ~~is~~ is 1 second.

5-letter search waited 500 ms and search of 10 or more letter waited 0.

It slows the request.

It matters becoz it makes short or empty searches slow for no reason

4) How I fixed it → I removed the artificial Thread.sleep() logic.

The API should process the request normally without intentionally blocking the server.

5) Before and After removing thread sleep() →

Before: |██████████| wait upto 1sec | - query - - |

After: | - query - - |

C) Bug 3: Bad input crashes the server (500)

1) where $\rightarrow$ File: TaskController.java

layer: Backend

Three inputs: status, page and pagesize

2) How did I found it $\rightarrow$ I tested the API with status invalid such as:
status = INVALID

The application throws an exception instead of proper client error.

I found the page problem by reading the code because it subtracts 1 from the page no.

3) Root cause $\rightarrow$ code uses: `TaskStatus.valueOf(status.toUpperCase())`

If the status does not exist in the enum, `valueOf` throws `IllegalArgumentException`

For page = 0, the start position becomes -1, which causes the crash of `sublist()`.

The start position is -ve so it crashes code and return 500.

4) How I fixed it $\rightarrow$ By adding validation try-catch block.

i) For Invalid status: the API returns 400
Bad request

ii) Page below 1 or page size outside 1 to 100:
returns 400

page = 0 $\rightarrow$ $(\text{page} - 1) \times 10$
 $(-1) \times 10 = -10$

5) Before and After

i) status = INVALID

ii) page = 0

iii) page = -1

iv) pageSize = 101

BEFORE

Exception
crash

crash

Accepted

AFTER

400 Bad req

400 Bad req

400 Bad req

400 Bad req

0] Bug 4 : Frontend: old responses and stuck loading

1) Where ~~to~~ File: useTasks.js
layer: Frontend

2) How did I found it $\rightarrow$ I read the hook and found that each request had no cleanup to cancel the previous search, status or page changed.
when error occurred loading was not reset.

3) Root cause $\rightarrow$ multiple req can be active at a same time. Without cancelling the previous req an older response can update the state after new req.
The loading state also needs to be reset after the request finishes.

4) How I fixed it $\rightarrow$ Each req now gets its own AbortController and useEffect cleanup function cancels the previous req when the search status or page changes. `api.js` passes the controller's signal to `fetch()` when req is cancelled, the code ignores the `AbortError` and does not update the state.

5) Before and After $\rightarrow$
Req A $\rightarrow$ slow
Req B $\rightarrow$ fast $\rightarrow$ UI
Req A $\rightarrow$ UI again

Req A $\rightarrow$ cancelled by
AbortController

Req B $\rightarrow$ latest res $\rightarrow$ UI

E] Bug 5: Frontend search typing and page reset

1) Where $\rightarrow$ File: App.jsx, useTasks.js
layer: frontend

2) How did I found it $\rightarrow$ I read the search and pagination code and found that every key press could trigger a request. I also found that changing the search or status could leave the user on the current page instead of resetting to page 1.

3) Root cause $\rightarrow$ Every key press sent a req to the server: eg typing api $\rightarrow$ it sends 3 req "a", "ap", "api"

The search text also needed to be separated from the value actually used for the API req.

4) How I fixed it $\rightarrow$ added debounce, so query is the text in search box, while debouncedQuery is updated 300ms after the last key press using setTimeout.

Add useEffect in App.jsx that calls setPage(1) whenever debouncedQuery or status changes.

This reduces unnecessary API req and ensures a new search or filter starts from page 1.

5) Before
"a" $\rightarrow$ server
"ap" $\rightarrow$ server
"api" $\rightarrow$ server
3 requests

After
"a" $\rightarrow$
"ap" $\rightarrow$
"api" $\rightarrow$
300 ms pause $\rightarrow$ server
1 request and goes to page 1